Experiment 4: Linear convolution, Circular Convolution, Correlation, and Autocorrelation

Consider two finite sequences x[n]=[1,3,-2,1,2,-1,4,4,2] and y[n] = [2,-1,4,1,-2,3]

4.1 Develop a program to compute the linear convolution of the two sequences after making their lengths equal through zero-padding, and compare the result with the inbuilt convolution function (time-domain equation).

```
import numpy as np

x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

length = max(len(x),len(y))

# Zero padding:-
x_pad = np.pad(x, (0,length-len(x)))
y_pad = np.pad(y, (0,length-len(y)))

print("Zero padded x[n]:",x_pad)
print("Zero padded y[n]:",y_pad)

# Convolution using in built function:-
output = np.convolve(y_pad,x_pad,mode='full')
print("\nConvolution using in built function:-")
print("Conv 1:",output)

# Convolution using for loops:-
L = len(x_pad) + len(y_pad) - 1
y_manual = np.zeros(L, dtype=int)

for n in range(L):
    for k in range(length):
        if 0 <= n - k <length:
            y_manual[n] += x_pad[k] * y_pad[n - k]

print("\nConvolution using time domain equation:-")
print("Conv2:",y_manual)
```

```
Zero padded x[n]: [ 1  3 -2  1  2 -1  4  4  2]
Zero padded y[n]: [ 2 -1  4  1 -2  3  0  0  0]

Convolution using in built function:-
Conv 1: [ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6  0  0  0]

Convolution using time domain equation:-
Conv2: [ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6  0  0  0]
```

4.2 Compute linear convolution when length of sequences are different and compare the results with that of inbuilt function.(Using equation of convolution in time-domain)

```
import numpy as np

x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

length = max(len(x),len(y))

# Convolution using in built function:-
output = np.convolve(y,x,mode='full')
print("\nConvolution using in built function:-")
print("Conv 1:",output)

z_manual = np.zeros(N, dtype=int)
# Convolution using for loops:-
L = len(x) + len(y) - 1
y_manual = np.zeros(L, dtype=int)

for n in range(length):
    for k in range(len(x)):
        if 0 <= n - k < len(y):
            z_manual[n] += x[k] * y[n - k]

print("\nConvolution using time domain equation:-")
print("Conv2:",z_manual)
```

```
Convolution using in built function:-
Conv 1: [ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Convolution using time domain equation:-
Conv2: [ 2  5 -3 17 -4 -5 31 -6 14]
```

4.3 Re-implement the above convolution problems using matrix formulations in the time domain.

```
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

Nx = len(x)
Ny = len(y)

# Output length
N = Nx + Ny - 1

# Create convolution matrix from x
X = np.zeros((N, Ny), dtype=int)
```

```python
for i in range(N):
    for j in range(Ny):
        if 0 <= i - j < Nx:
            X[i, j] = x[i - j]

# Matrix multiplication
z_matrix = X @ y

# Inbuilt convolution
z_builtin = np.convolve(x, y)

# Display convolution matrix
print("Convolution Matrix X:")
print(X)

print("\nResult using Matrix Formulation:")
print(z_matrix)

print("\nResult using np.convolve():")
print(z_builtin)

# Compare results
if np.array_equal(z_matrix, z_builtin):
    print("\nBoth results are identical.")
else:
    print("\nResults are different.")
```

```
Convolution Matrix X:
[[ 1  0  0  0  0  0]
 [ 3  1  0  0  0  0]
 [-2  3  1  0  0  0]
 [ 1 -2  3  1  0  0]
 [ 2  1 -2  3  1  0]
 [-1  2  1 -2  3  1]
 [ 4 -1  2  1 -2  3]
 [ 4  4 -1  2  1 -2]
 [ 2  4  4 -1  2  1]
 [ 0  2  4  4 -1  2]
 [ 0  0  2  4  4 -1]
 [ 0  0  0  2  4  4]
 [ 0  0  0  0  2  4]
 [ 0  0  0  0  0  2]]

Result using Matrix Formulation:
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Result using np.convolve():
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Both results are identical.
```

4.4 Compute linear convolution of x and y using dot product function.

```
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

# Length of convolution output
N = len(x) + len(y) - 1

# Array for storing result
z = np.zeros(N, dtype=int)

# Linear convolution using dot product
for n in range(N):

    # Create overlapping portions
    x_start = max(0, n - len(y) + 1)
    x_end = min(n + 1, len(x))

    y_start = max(0, n - len(x) + 1)
    y_end = min(n + 1, len(y))

    # Corresponding portions
    x_part = x[x_start:x_end]
    y_part = y[y_start:y_end][::-1]

    # Dot product
    z[n] = np.dot(x_part, y_part)

# Built-in convolution
z_builtin = np.convolve(x, y)

# Display results
print("x[n] =", x)
print("y[n] =", y)

print("\nLinear convolution using dot product:")
print(z)

print("\nLinear convolution using np.convolve():")
print(z_builtin)

# Compare results
if np.array_equal(z, z_builtin):
    print("\nBoth results are identical.")
else:
    print("\nResults are different.")
```

```
x[n] = [ 1  3 -2  1  2 -1  4  4  2]
y[n] = [ 2 -1  4  1 -2  3]

Linear convolution using dot product:
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Linear convolution using np.convolve():
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Both results are identical.
```

4.5 Write a program to compute circular convolution of x and y using time-domain approach using (a) basic equation and (b) matrix operations.

```python
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

# Make both sequences equal in length
N = max(len(x), len(y))

x_pad = np.pad(x, (0, N - len(x)))
y_pad = np.pad(y, (0, N - len(y)))

# Circular convolution using basic equation
z_basic = np.zeros(N, dtype=int)

for n in range(N):
    for k in range(N):
        z_basic[n] += x_pad[k] * y_pad[(n - k) % N]

print("x[n] =", x_pad)
print("y[n] =", y_pad)

print("\nCircular convolution using basic equation:")
print(z_basic)
```

```
x[n] = [ 1  3 -2  1  2 -1  4  4  2]
y[n] = [ 2 -1  4  1 -2  3  0  0  0]

Circular convolution using basic equation:
[28  6  3 25  2 -5 31 -6 14]
```

```python
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])
```

```python
# Make lengths equal
N = max(len(x), len(y))

x_pad = np.pad(x, (0, N - len(x)))
y_pad = np.pad(y, (0, N - len(y)))

# Create circulant matrix
X = np.zeros((N, N), dtype=int)

for i in range(N):
    for k in range(N):
        X[i, k] = x_pad[(i - k) % N]

# Matrix multiplication
z_matrix = X @ y_pad

print("x[n] =", x_pad)
print("y[n] =", y_pad)

print("\nCirculant Matrix:")
print(X)

print("\nCircular convolution using matrix operation:")
print(z_matrix)

# Compare both methods
z_basic = np.zeros(N, dtype=int)

for n in range(N):
    for k in range(N):
        z_basic[n] += x_pad[k] * y_pad[(n - k) % N]

print("\nCircular convolution using basic equation:")
print(z_basic)

if np.array_equal(z_basic, z_matrix):
    print("\nBoth results are identical.")
else:
    print("\nResults are different.")
```

```
x[n] = [ 1  3 -2  1  2 -1  4  4  2]
y[n] = [ 2 -1  4  1 -2  3  0  0  0]

Circulant Matrix:
[[ 1  2  4  4 -1  2  1 -2  3]
 [ 3  1  2  4  4 -1  2  1 -2]
 [-2  3  1  2  4  4 -1  2  1]
 [ 1 -2  3  1  2  4  4 -1  2]
 [ 2  1 -2  3  1  2  4  4 -1]
 [-1  2  1 -2  3  1  2  4  4]
 [ 4 -1  2  1 -2  3  1  2  4]
```

```
 [ 4  4 -1  2  1 -2  3  1  2]
 [ 2  4  4 -1  2  1 -2  3  1]]

Circular convolution using matrix operation:
[28  6  3 25  2 -5 31 -6 14]

Circular convolution using basic equation:
[28  6  3 25  2 -5 31 -6 14]

Both results are identical.
```

4.6 Compute linear convolution of x and y using using circular convolution in time domain.

```python
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

# Length required for linear convolution
N = len(x) + len(y) - 1

# Zero-padding both sequences to N
x_pad = np.pad(x, (0, N - len(x)))
y_pad = np.pad(y, (0, N - len(y)))

# Circular convolution using time-domain equation
z_circular = np.zeros(N, dtype=int)

for n in range(N):
    for k in range(N):
        z_circular[n] += x_pad[k] * y_pad[(n - k) % N]

# Direct linear convolution for comparison
z_linear = np.convolve(x, y)

# Display results
print("Zero-padded x[n]:")
print(x_pad)

print("\nZero-padded y[n]:")
print(y_pad)

print("\nLinear convolution using circular convolution:")
print(z_circular)

print("\nBuilt-in linear convolution:")
print(z_linear)

# Compare results
```

```
if np.array_equal(z_circular, z_linear):
    print("\nBoth results are identical.")
else:
    print("\nResults are different.")
```

```
Zero-padded x[n]:
[ 1  3 -2  1  2 -1  4  4  2  0  0  0  0  0]

Zero-padded y[n]:
[ 2 -1  4  1 -2  3  0  0  0  0  0  0  0  0]

Linear convolution using circular convolution:
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Built-in linear convolution:
[ 2  5 -3 17 -4 -5 31 -6 14 26  1  6  8  6]

Both results are identical.
```

4.7 Write a program to compute circular convolution of x and y using frequency-domain approach.

```
import numpy as np

# Given sequences
x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

# Length for circular convolution
N = max(len(x), len(y))

# Zero-pad the shorter sequence
x_pad = np.pad(x, (0, N - len(x)))
y_pad = np.pad(y, (0, N - len(y)))

# DFT of both sequences
X = np.fft.fft(x_pad)
Y = np.fft.fft(y_pad)

# Multiplication in frequency domain
Z = X * Y

# IDFT to obtain circular convolution
z_freq = np.fft.ifft(Z)

# Remove very small numerical errors
z_freq = np.real_if_close(z_freq)
z_freq = np.round(z_freq).astype(int)

# Direct circular convolution for comparison
```

```
z_time = np.zeros(N, dtype=int)

for n in range(N):
    for k in range(N):
        z_time[n] += x_pad[k] * y_pad[(n - k) % N]

# Display results
print("x[n] =", x_pad)
print("y[n] =", y_pad)

print("\nDFT of x[n]:")
print(np.round(X, 2))

print("\nDFT of y[n]:")
print(np.round(Y, 2))

print("\nCircular convolution using frequency-domain approach:")
print(z_freq)

print("\nCircular convolution using time-domain approach:")
print(z_time)

# Compare results
if np.array_equal(z_freq, z_time):
    print("\nBoth results are identical.")
else:
    print("\nResults are different.")
```

```
x[n] = [ 1  3 -2  1  2 -1  4  4  2]
y[n] = [ 2 -1  4  1 -2  3  0  0  0]

DFT of x[n]:
[14.  +0.j    1.74+6.84j -1.75+0.4j   2.  -8.66j -4.49+1.35j -4.49-
1.35j
  2.  +8.66j -1.75-0.4j   1.74-6.84j]

DFT of y[n]:
[ 7.  +0.j    0.49-2.45j -1.67-2.73j  1.  +8.66j  5.68-2.88j
5.68+2.88j
  1.  -8.66j -1.67+2.73j  0.49+2.45j]

Circular convolution using frequency-domain approach:
[28  6  3 25  2 -5 31 -6 14]

Circular convolution using time-domain approach:
[28  6  3 25  2 -5 31 -6 14]

Both results are identical.
```

4.8 Compare the time-complexities of the above methods.

```
import numpy as np
import time

x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

def linear_convolution(x, y):
    N = len(x) + len(y) - 1
    z = np.zeros(N)

    for n in range(N):
        for k in range(len(x)):
            if 0 <= n-k < len(y):
                z[n] += x[k] * y[n-k]

    return z


def circular_convolution(x, y):
    N = max(len(x), len(y))

    x = np.pad(x, (0, N-len(x)))
    y = np.pad(y, (0, N-len(y)))

    z = np.zeros(N)

    for n in range(N):
        for k in range(N):
            z[n] += x[k] * y[(n-k) % N]

    return z


def circular_convolution_fft(x, y):
    N = max(len(x), len(y))

    x = np.pad(x, (0, N-len(x)))
    y = np.pad(y, (0, N-len(y)))

    X = np.fft.fft(x)
    Y = np.fft.fft(y)

    return np.real(np.fft.ifft(X * Y))


start = time.perf_counter()
linear_convolution(x, y)
t1 = time.perf_counter() - start

start = time.perf_counter()
circular_convolution(x, y)
```

```
t2 = time.perf_counter() - start

start = time.perf_counter()
circular_convolution_fft(x, y)
t3 = time.perf_counter() - start

print("Time complexities:")
print("Linear convolution (time domain)      : O(N^2)")
print("Circular convolution (time domain)   : O(N^2)")
print("Circular convolution (matrix method)  : O(N^2)")
print("Linear convolution (dot product)      : O(N^2)")
print("Circular convolution (FFT)            : O(N log N)")

print("\nExecution times:")
print("Linear convolution:", t1, "seconds")
print("Circular convolution:", t2, "seconds")
print("Circular convolution using FFT:", t3, "seconds")
```

```
Time complexities:
Linear convolution (time domain)      : O(N^2)
Circular convolution (time domain)   : O(N^2)
Circular convolution (matrix method)  : O(N^2)
Linear convolution (dot product)      : O(N^2)
Circular convolution (FFT)            : O(N log N)

Execution times:
Linear convolution: 0.00010959999963233713 seconds
Circular convolution: 0.0002133999996658531 seconds
Circular convolution using FFT: 0.0005025999998906627 seconds
```

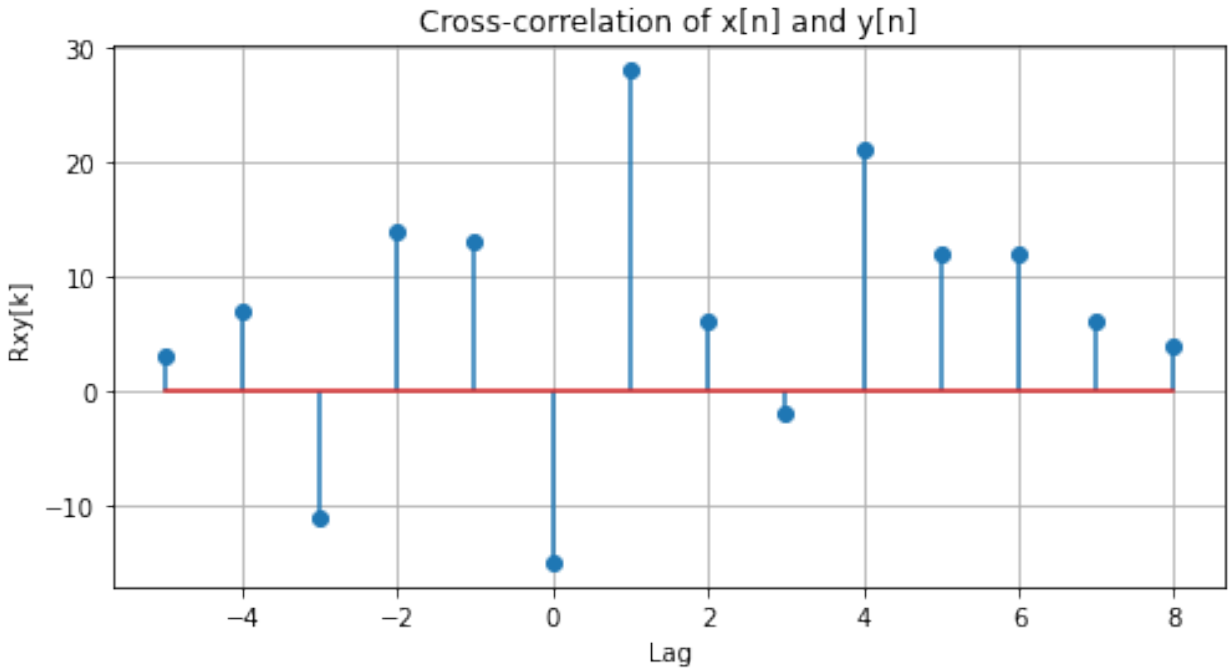
4.9 (a)Dertermine and plot crosscorrelation and autocorelation sequences for the above sequences. (b)Add a random AWGN noise b[n] to x[n] and plot the autocoreelation. Comment on the results. (c)Plot autocorrelation of the AWGN noise signal.Comment on the results. (d)Compute cross correlation of x[n] and y[n], where y[n]= x[n-N]. Comment on the results. Note: Use given x[n] and y[n] for all the problem statements above.

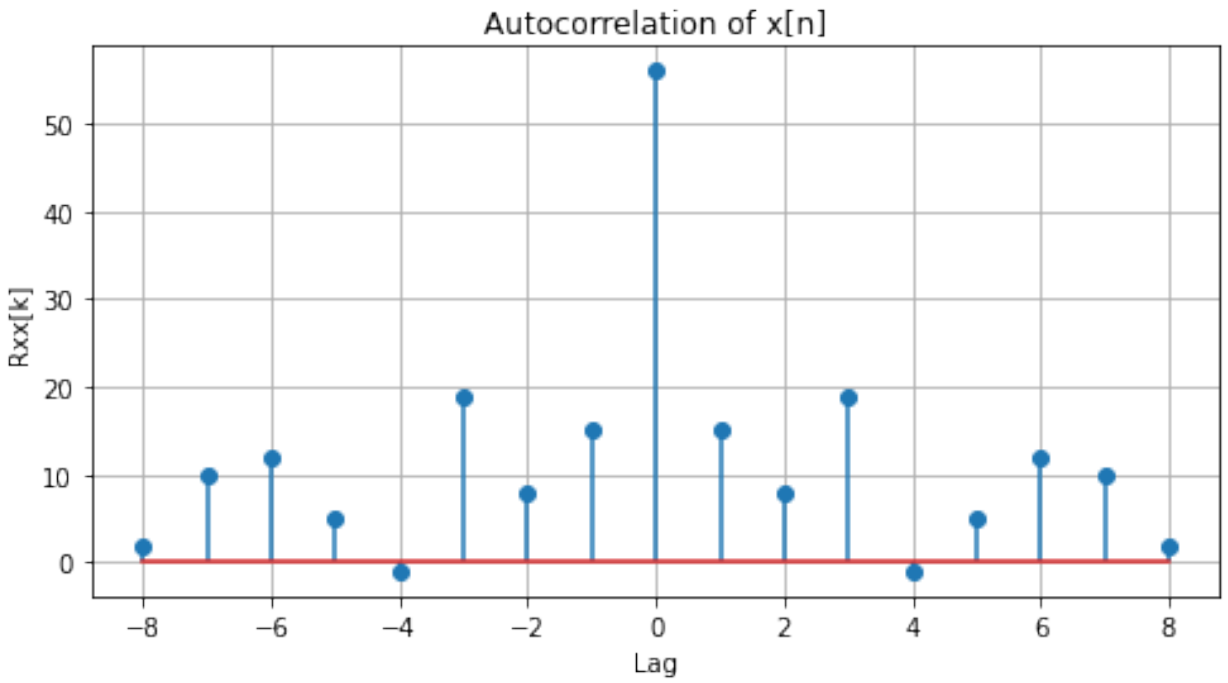
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 3, -2, 1, 2, -1, 4, 4, 2])
y = np.array([2, -1, 4, 1, -2, 3])

cross_corr = np.correlate(x, y, mode='full')
auto_corr_x = np.correlate(x, x, mode='full')
auto_corr_y = np.correlate(y, y, mode='full')

lags_xy = np.arange(-(len(y)-1), len(x))
lags_x = np.arange(-(len(x)-1), len(x))
lags_y = np.arange(-(len(y)-1), len(y))
```

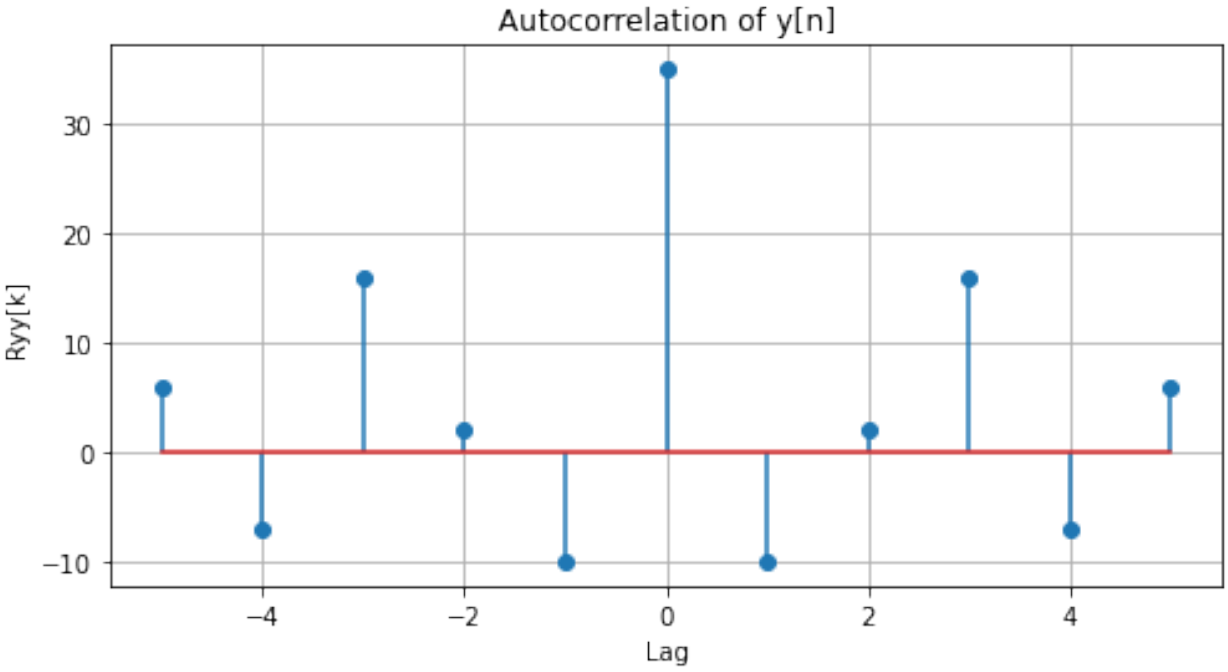
```
print("Cross-correlation of x and y:")
print(cross_corr)

print("\nAutocorrelation of x:")
print(auto_corr_x)

print("\nAutocorrelation of y:")
print(auto_corr_y)

plt.figure(figsize=(8, 4))
plt.stem(lags_xy, cross_corr)
plt.title("Cross-correlation of x[n] and y[n]")
plt.xlabel("Lag")
plt.ylabel("Rxy[k]")
plt.grid()
plt.show()

plt.figure(figsize=(8, 4))
plt.stem(lags_x, auto_corr_x)
plt.title("Autocorrelation of x[n]")
plt.xlabel("Lag")
plt.ylabel("Rxx[k]")
plt.grid()
plt.show()

plt.figure(figsize=(8, 4))
plt.stem(lags_y, auto_corr_y)
plt.title("Autocorrelation of y[n]")
plt.xlabel("Lag")
plt.ylabel("Ryy[k]")
plt.grid()
plt.show()


np.random.seed(10)

noise_power = 2
noise = np.sqrt(noise_power) * np.random.randn(len(x))

x_noisy = x + noise

auto_corr_noisy = np.correlate(x_noisy, x_noisy, mode='full')

print("\nAWGN:")
print(np.round(noise, 3))

print("\nNoisy x[n]:")
print(np.round(x_noisy, 3))

print("\nAutocorrelation of noisy x[n]:")
```

```
print(np.round(auto_corr_noisy, 3))

lags_noisy = np.arange(-(len(x)-1), len(x))

plt.figure(figsize=(8, 4))
plt.stem(lags_noisy, auto_corr_noisy)
plt.title("Autocorrelation of x[n] + AWGN")
plt.xlabel("Lag")
plt.ylabel("Rxx[k]")
plt.grid()
plt.show()


auto_corr_noise = np.correlate(noise, noise, mode='full')

print("\nAutocorrelation of AWGN:")
print(np.round(auto_corr_noise, 3))

plt.figure(figsize=(8, 4))
plt.stem(lags_noisy, auto_corr_noise)
plt.title("Autocorrelation of AWGN Noise")
plt.xlabel("Lag")
plt.ylabel("Rnn[k]")
plt.grid()
plt.show()


N = 3

y_delay = np.zeros(len(x), dtype=int)
y_delay[N:] = x[:-N]

cross_corr_delay = np.correlate(x, y_delay, mode='full')

lags_delay = np.arange(-(len(y_delay)-1), len(x))

print("\nDelayed y[n] = x[n-N]:")
print(y_delay)

print("\nCross-correlation of x[n] and x[n-N]:")
print(cross_corr_delay)

plt.figure(figsize=(8, 4))
plt.stem(lags_delay, cross_corr_delay)
plt.title("Cross-correlation of x[n] and x[n-N]")
plt.xlabel("Lag")
plt.ylabel("Rxy[k]")
plt.grid()
plt.show()
```

```
Cross-correlation of x and y:
[  3   7 -11  14  13 -15  28   6  -2  21  12  12   6   4]

Autocorrelation of x:
[ 2 10 12  5 -1 19  8 15 56 15  8 19 -1  5 12 10  2]

Autocorrelation of y:
[  6  -7  16   2 -10  35 -10   2  16  -7   6]
```

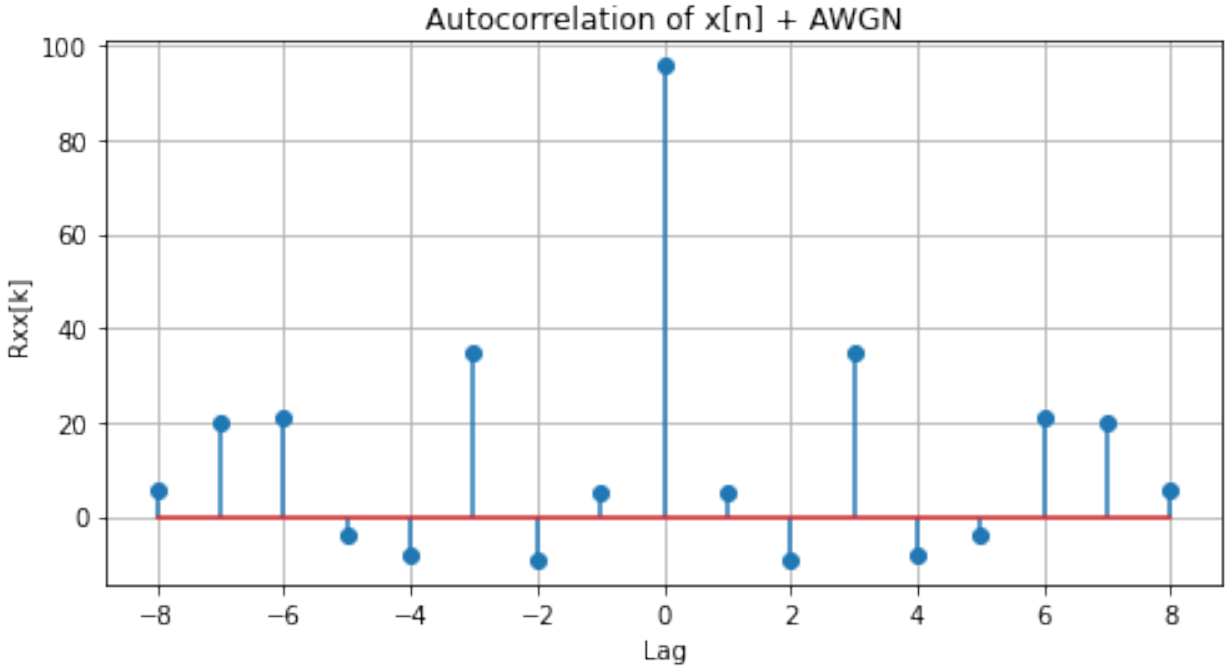




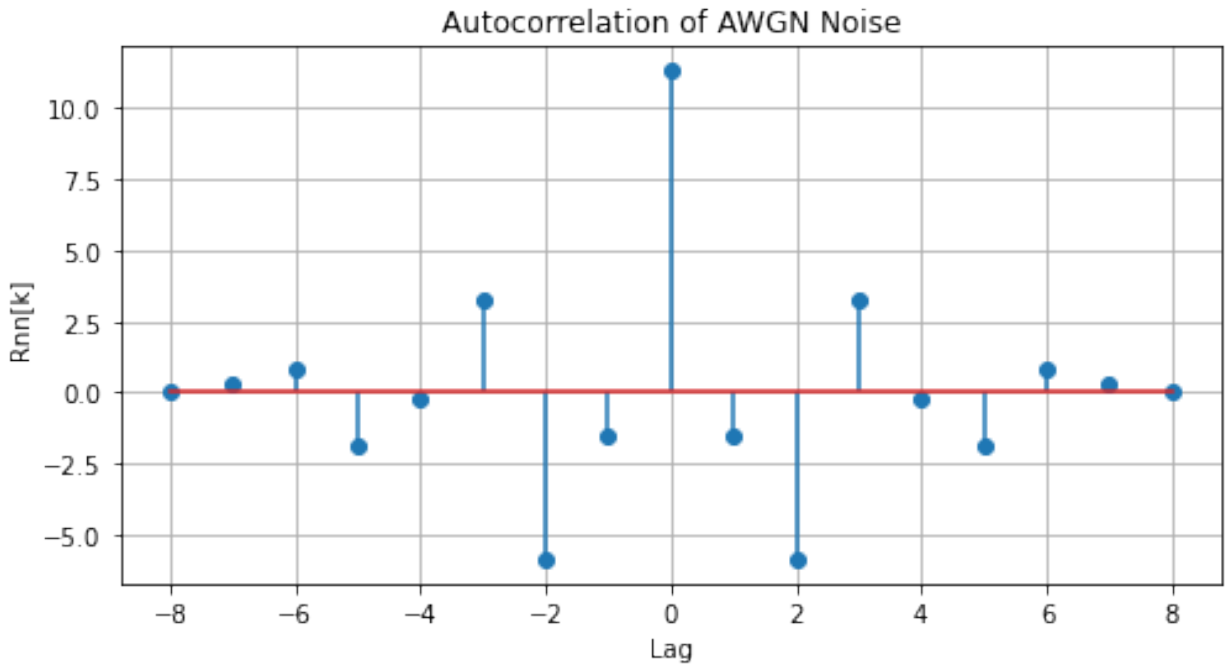



```
AWGN:
[ 1.883  1.012 -2.186 -0.012  0.879 -1.018  0.375  0.154  0.006]

Noisy x[n]:
[ 2.883  4.012 -4.186  0.988  2.879 -2.018  4.375  4.154  2.006]
```

```
Autocorrelation of noisy x[n]:
[ 5.784 20.023 20.881 -3.669 -8.232 35.076 -9.157  5.348 95.682  5.348
 -9.157 35.076 -8.232 -3.669 20.881 20.023  5.784]
```




```
Autocorrelation of AWGN:
[ 1.100e-02  2.950e-01  8.490e-01 -1.873e+00 -1.930e-01  3.216e+00
 -5.860e+00 -1.509e+00  1.132e+01 -1.509e+00 -5.860e+00  3.216e+00
 -1.930e-01 -1.873e+00  8.490e-01  2.950e-01  1.100e-02]
```




```
Delayed y[n] = x[n-N]:
[ 0  0  0  1  3 -2  1  2 -1]

Cross-correlation of x[n] and x[n-N]:
[-1 -1  9 -4 -5 20 -9  0 19 -1  5 12 10  2  0  0  0]
```